Big Data
(MEI)

João Pedro Costa
jcosta@isec.pt

1

Agenda

- Hive Storage Formats
- Cluster Urls
- Hive JDBC connection

2

CREATE TABLE … STORED AS …



Table Storage Formats

File Type	Format	Compression	Can INSERT
TEXTFILE	Unstructured	LZO, gzip, bzip2, Snappy default file format is uncompressed text, with values separated by ASCII 0x01 characters	Yes: CREATE TABLE, INSERT, LOAD DATA, and query. If LZO compression is used, you must create the table and load data in Hive. If other kinds of compression are used, you must load data through LOAD DATA, Hive, or manually in HDFS.
PARQUET	Structured	Snappy, gzip	Yes: CREATE TABLE, INSERT, LOAD DATA, and query.
AVRO	Structured	Snappy, gzip, deflate, bzip2	No. Import data by using LOAD DATA on data files already in the right format,
RCFILE	Structured	Snappy, gzip, deflate, bzip2	No. Import data by using LOAD DATA on data files already in the right format
SEQUENCEFILE	Structured	Snappy, gzip, deflate, bzip2	No. Import data by using LOAD DATA on data files already in the right format
ORC	Structured	Snappy, gzip,	Yes: CREATE TABLE, INSERT, LOAD DATA, and query.

3

Flat Files



- access to structured file types and plain text files (flat files)
- data is stored in lines, with each line being a record.
- each line is terminated by a newline character `'\n'`
- typical types
  - CSV files (comma-separated values) ‘,' or‘;'
  - TSV files (tab-separated values) ‘\t'
  - PSV files (pipe-separated values) '|'

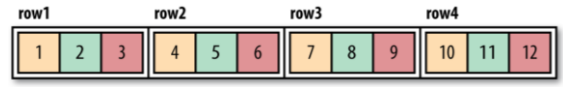
```
CREATE TABLE rcfile_table (
    first_name STRING,
    last_name  STRING     )
 STORED AS TEXTFILE;
```

```
CREATE TABLE rcfile_table (
    first_name STRING,
    last_name  STRING     )
 STORED AS TEXTFILE
 FIELDS TERMINATED BY '|';
```


4

SequenceFile

- a flat file consisting of binary key/value pairs
- in binary format and these files are able to split
- main advantage is to merge two or more files into one file
- extensively used in MapReduce as input/output formats
- internally, the temporary outputs of maps are stored using SequenceFile




```
CREATE TABLE rcfile_table (
    first_name STRING,
    last_name  STRING     )
 STORED AS SEQUENCEFILE;
```


5

SequenceFile

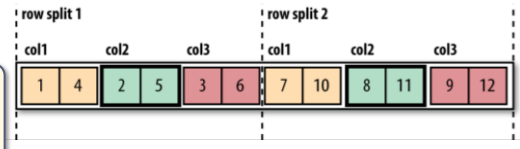
- provides a **`Writer`**, **`Reader`** and **`Sorter`** classes for writing, reading and sorting
- there are 3 different SequenceFile formats:
  - uncompressed key/value records
  - record compressed key/value records - only 'values' are compressed
  - block compressed key/value records - both keys and values are collected in 'blocks' separately and compressed
- **`SequenceFile.Reader`** acts as a bridge and can read any of the above SequenceFile formats
- recommended the use of **`SequenceFile.createWriter`** methods to construct the 'preferred' writer implementation

6

RCFILE - Record Columnar File

- a data placement structure for MapReduce-based data warehouse systems
- stores table data in a flat file consisting of binary key/value pairs
- partitions rows horizontally into row splits, and then partitions vertically each row split in a columnar way
- stores the metadata of a row split as the key part of a record, and all the data of a row split as the value part

```
CREATE TABLE rcfile_table (
    first_name STRING,
    last_name  STRING
 )
 STORED AS RCFILE;
```



RCFILE - Record Columnar File

- combines the advantages of both row-store and column-store to satisfy the need for fast data loading and query processing, efficient use of storage space, and adaptability to highly dynamic workload patterns
- as row-store, RCFile guarantees that data in the same row are located in the same node
- as column-store, RCFile can exploit column-wise data compression and skip unnecessary column reads

AVRO



- provides data serialization and data exchange services for Hadoop
- these services can be used together or independently
- a schema-based system
  - language-independent schema associated with its read and write operations
- is a data serialization system
  - bigdata can be exchanged between programs written in any language
- is the robust support for data schemas that change over time

```
CREATE TABLE avro_table (
    first_name STRING,
    last_name  STRING
)
STORED AS AVRO;
```


AVRO



Avro Provides:

- a compact, fast, binary data format
- a container file, to store persistent data
- remote procedure call (RPC).
- simple integration with dynamic languages

AVRO



Simple Avro Schema

- created in JavaScript Object Notation (JSON) document format,
- is a lightweight text-based data interchange format

Example:

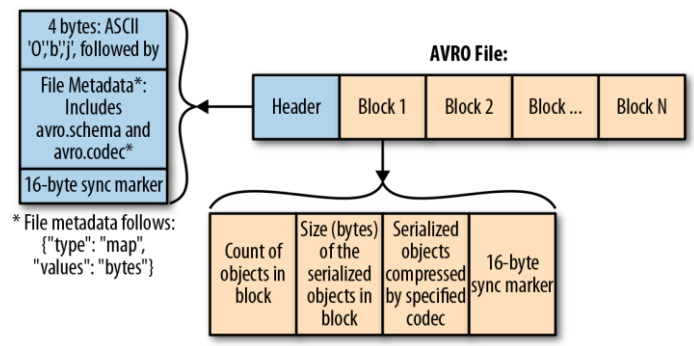
*schema defines a (**`record`** type) document within "**`AvroSample`**" namespace*

*name of document is "**`Employee`**" which contains two "**`fields`**" → **`name`** and **`age`***

```
{
   "type" : "record",
   "namespace" : "AvroSample",
   "name" : "Employee",
   "fields" : [
      { "name" : "Name" , "type" : "string" },
      { "name" : "age"  , "type" : "int" }
   ]
}
```

`type` – describes document type
`namespace` – describes the name of the namespace in which the object resides
`name` – describes the schema name
`fields` – attribute array containing
  `name` – describes the name of field
  `type` – describes data type of field

AVRO



PARQUET

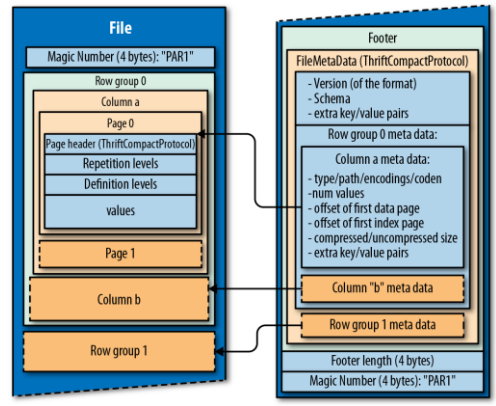
- a columnar storage format in the Hadoop ecosystem
- stores binary data in a column-oriented way, where the values of each column are organized so that they are all adjacent
  - enables better compression
- more efficient in storage and has better query performance
  - compared to a traditional row-oriented format
- has the following important characteristics:
  - self-describing data format
  - columnar format
  - flexible compression options
  - large file size

```
CREATE TABLE parquet_table (
    first_name STRING,
    last_name  STRING
 )
 STORED AS PARQUET;
```

Cloudera & Twitter

PARQUET

- **Row groups**
  - horizontal partitions of the data
- **Column chunks**
  - data for a single column within a row group
- **Pages**
  - smallest storage unit (where encoding/compression happens)
- **Metadata footer**
  - schema
  - statistics (min/max values, etc.)



Cloudera & Twitter

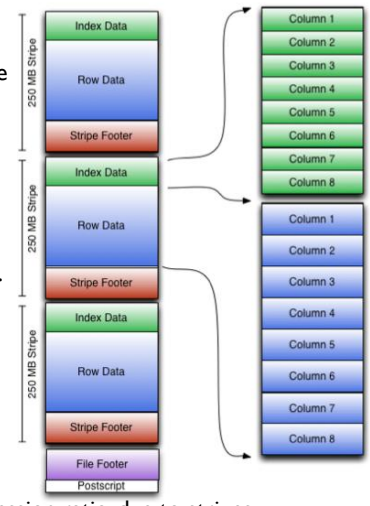
ORC - *Optimized Row Columnar*

- provides a highly efficient way to store Hive data
- optimized reading, writing, and processing data in Hive
- advantages over RCFile format:
  - a single file as the output of each task, which reduces NameNode's load
  - Hive type support including datetime, decimal, and the complex types (struct, list, map, and union)
  - lightweight indexes stored within the file
    - skip row groups that don't pass predicate filtering seek to a given row
  - block-mode compression based on data type
    - run-length encoding for integer columns
    - dictionary encoding for string columns
  - concurrent reads of the same file using separate RecordReaders
  - ability to split files without scanning for markers
  - bound the amount of memory needed for reading or writing

HortonWorks

ORC - *Optimized Row Columnar*

- contains groups of row data called **stripes**, along with auxiliary information in a **file footer**
- **postscript** holds compression parameters and the size of the compressed footer
- file footer contains a list of stripes, the number of rows per stripe, and each column's data type.
- **stripe footer** contains a directory of stream locations.
- **Index data** includes min and max values for each column and the row positions within each column.
- **Row index** entries provide offsets that enable seeking to the right compression block and byte within a decompressed block.
- ORC indexes are used only for stripes selection and row groups and not for answering queries
- row index entries enables row-skipping within a stripe for rapid reads, despite large stripe sizes



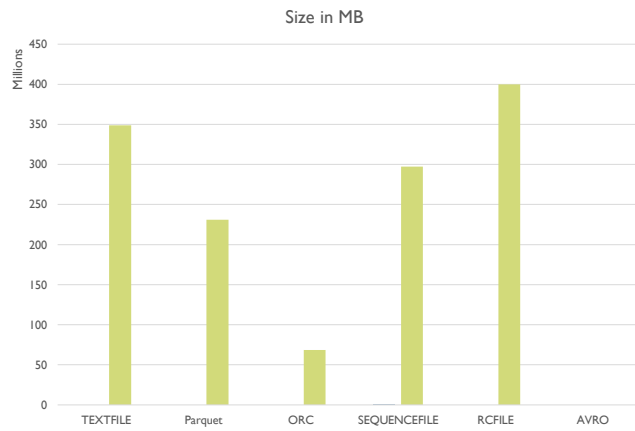
Best compression ratio, due to stripes

ORC - *Optimized Row Columnar*

```
CREATE TABLE orc_table (
    first_name STRING,
    last_name  STRING
 )
 STORED AS ORC;
```


20

Size Comparison



22

ORC, Parquet, Avro

ORC, Parquet, and Avro			
	ORC	**Parquet**	**Avro**
Row or column	Column	Column	Row
Compression	Great	Great	Good
Speedup (compared to text file)	10–100x	10–100x	10x
Schema evolution	Good	Better	Best
Platforms	Hive, Spark, Presto	Hive, Spark, Presto	Hive, Spark
File statistics	Yes	Yes	No
Indexes	Yes	Yes	No
Bloom filters	Yes	No	No

Desafio #2



Periodicamente as lojas de uma cadeia de retalho enviam um ficheiro com as vendas que registou. Considere que as vendas, num período, da loja A estão registadas no ficheiro http://10.84.128.2/jcosta/bigdata/dataset/retail0/retail_A_1.csv

a) Torne os dados do ficheiro acessível para pesquisado através do Hive?

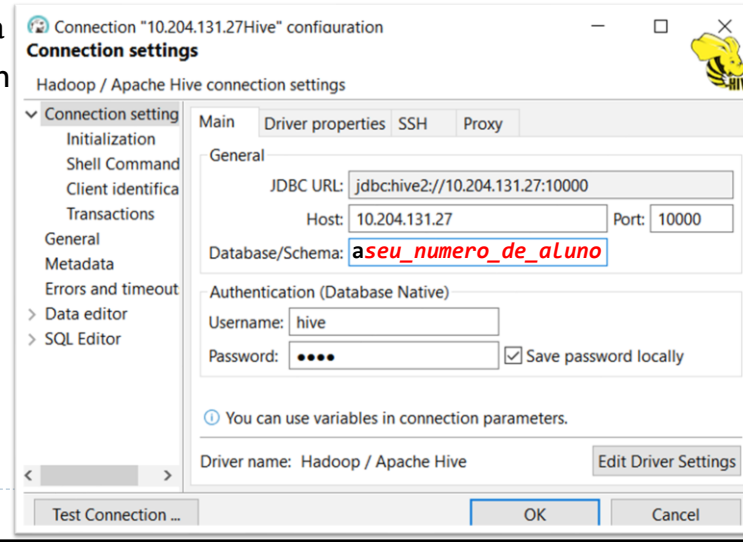
b) Faça o desafio#2 no moodle https://moodle.isec.pt/moodle/mod/quiz/view.php?id=213166

```
# Create table retail
create table retail (
    invoice           int,
    stock_code        string,
    description       string,
    quantity          int,
    invoice_date      string,
    price             string,
    customer_id       int,
    country           string)
row format delimited fields terminated by ';'
tblproperties("skip.header.line.count"="1");

# load the data from file system
load data local inpath '/user/retail_A_1.csv‘ into table retail;
```

JDBC Connection

- Use Dbeaver (https://dbeaver.io/download/)
- Create a connection



25

Cluster

- **HDFS** http://10.84.129.52:9870
- **Datanode** http://10.84.129.52:9864
- **YARN App** http://10.84.129.52:8088
- **YARN Hist** http://10.84.129.52:8188
- **SPARK** http://10.84.128.47:8080
- **SPARKjobs** http://10.84.128.47:4040

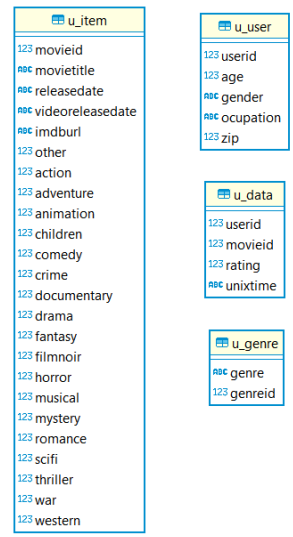
- **HIVE JDBC Connection**
  jdbc:hive2://10.84.128.48:10000/FIRST_LAST
  username: FIRST_LAST password: any

  Pasta no hdfs da base de dados
  /user/hive/warehouse/FIRST_LAST.db

26

MovieLens database

- movie ratings.
- Stable benchmark dataset
- 100,000 ratings
- from 1000 users
- on 1700 movies
- Released 4/1998.



http://files.grouplens.org/datasets/movielens/ml-100k.zip

27

MovieLens database

```
apt update
apt install wget unzip

wget http://files.grouplens.org/datasets/movielens/ml-100k.zip

unzip ml-100k.zip

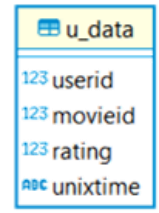

hive
```


28

MovieLens database




```
CREATE TABLE u_data (
     userid     INT,
     movieid    INT,
     rating     INT,
     unixtime   INT)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t'
STORED AS TEXTFILE;
```




```
LOAD DATA INPATH '/input/movielens/u.data'
OVERWRITE INTO TABLE u_data;
```


29

MovieLens database




```
CREATE TABLE u_item (
     movieid      int,      movietitle       string,
     releasedate  string,   videoreleasedate string,
     IMDbURL      string,   other            tinyint,
     Action       tinyint,  Adventure        tinyint,
     Animation    tinyint,  Children         tinyint,
     Comedy       tinyint,  Crime            tinyint,
     Documentary  tinyint,  Drama            tinyint,
     Fantasy      tinyint,  FilmNoir         tinyint,
     Horror       tinyint,  Musical          tinyint,
     Mystery      tinyint,  Romance          tinyint,
     SciFi        tinyint,  Thriller         tinyint,
     War          tinyint,  Western          tinyint )
 ROW FORMAT DELIMITED FIELDS TERMINATED BY '|'
 STORED AS TEXTFILE;

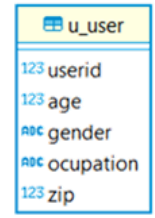
LOAD DATA INPATH '/input/movielens/u.item'
OVERWRITE INTO TABLE u_item;
```



30

MovieLens database

```
CREATE TABLE u_user (
    userid      INT,
    age         INT,
    gender      char(1),
    ocupation   string,
    zip         INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '|'
STORED AS TEXTFILE;
```




```
LOAD DATA INPATH '/input/movielens/u.user'
OVERWRITE INTO TABLE u_user;
```


MovieLens database

```
CREATE TABLE u_genre (
    genre    string,
    genreid  INT)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '|'
STORED AS TEXTFILE;
```




```
LOAD DATA INPATH '/input/movielens/u.genre'
OVERWRITE INTO TABLE u_genre;
```

MovieLens database



- calcular o rating médio e o número comentários de cada filme (e o rating ponderado)
- top 5 dos filmes com maior número de rating (5)
- Filme com mais comentários

Desafio #3



- Procurar um dataset (> 1GB)
- Load de uma tabela grande para os formatos:
  - TEXTFILE
  - SEQUENCEFILE
  - AVRO
  - RCFILE
  - PARQUET

Avaliação comparativa :

- do espaço ocupado
- tempos de execução de 2 queries (definir)
  - Query simples
  - Query com group by

Desafio #3 Conclusions

- ??